

■ LEVEL 13 • BACKTRACKING

Pattern Recognition • Problem Practice • Optimal C++ • Complexity

Pattern Trigger	Generate valid choices/configurations. Choose → Explore → Undo.
Reference Examples	Subsets • Permutations • Combinations • Combination Sum • N-Queens • Sudoku
Problems	6 focused problems in this level
Code Standard	C++ • visible nested indentation • syntax-colored keywords/functions/comments/strings

Reference framework: the user's **DSA Approach Handbook**. The handbook defines the trigger, why, key thought, complexity, and representative practice problems; this continuation turns each level into a problem-by-problem practice book.

LEVEL 13 • EXPANDED PROBLEM INDEX

The original 6 core problems are retained. Four additional problems are added below, giving 10 total problems in this level.

#	Problem	Pattern	LeetCode
1	Subsets	Backtracking	#78
2	Permutations	Backtracking	#46
3	Combination Sum	Backtracking	#39
4	Generate Parentheses	Backtracking	#22
5	N-Queens	Backtracking	#51
6	Word Search	Backtracking on Grid	#79
7	Subsets	Backtracking	78
8	Permutations	Backtracking	46
9	Combination Sum	Backtracking	39
10	Letter Combinations of a Phone Number	Choice Tree Backtracking	17

Extension rule: Problems 7–10 are additional practice chosen to strengthen the same pattern. They are clearly separated from the reference handbook's original representative set.

PROBLEM 1 Subsets

Pattern: Backtracking

LeetCode: #78

■ Input	array of distinct integers
■ Output	all subsets
■ Constraints	$n \leq 10$
■ Main Idea / Pattern	At each index, choose the element or skip it.
■ Time Complexity	$O(n \cdot 2^n)$
■ Space Complexity	$O(n)$ excluding output

■ Optimal C++ Code

```
void dfs(int i, vector<int>& nums, vector<int>& cur, vector<vector<int>>& ans) {
    if (i == nums.size()) {
        ans.push_back(cur);
        return;
    }

    dfs(i + 1, nums, cur, ans);

    cur.push_back(nums[i]);
    dfs(i + 1, nums, cur, ans);
    cur.pop_back();
}

vector<vector<int>> subsets(vector<int>& nums) {
    vector<vector<int>> ans, cur;
    dfs(0, nums, cur, ans);
    return ans;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 2 Permutations

Pattern: Backtracking

LeetCode: #46

■ Input	distinct integers
■ Output	all permutations
■ Constraints	$n \leq 6$
■ Main Idea / Pattern	Choose an unused element, explore, then undo the choice.
■ Time Complexity	$O(n \cdot n!)$
■ Space Complexity	$O(n)$ excluding output

■ Optimal C++ Code

```
void dfs(vector<int>& nums, vector<bool>& used,
        vector<int>& cur, vector<vector<int>>& ans) {
    if (cur.size() == nums.size()) {
        ans.push_back(cur);
        return;
    }

    for (int i = 0; i < nums.size(); i++) {
        if (used[i]) continue;

        used[i] = true;
        cur.push_back(nums[i]);

        dfs(nums, used, cur, ans);

        cur.pop_back();
        used[i] = false;
    }
}

vector<vector<int>> permute(vector<int>& nums) {
    vector<vector<int>> ans;
    vector<int> cur;
    vector<bool> used(nums.size());
    dfs(nums, used, cur, ans);
    return ans;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 3 Combination Sum

Pattern: Backtracking

LeetCode: #39

■ Input	distinct positive candidates and target
■ Output	all combinations summing to target
■ Constraints	a number can be reused
■ Main Idea / Pattern	Choose a candidate, recurse with the same index, then undo.
■ Time Complexity	$O(N^{(T/M)})$ roughly
■ Space Complexity	$O(T/M)$ recursion

■ Optimal C++ Code

```
void dfs(int start, int target, vector<int>& a,
        vector<int>& cur, vector<vector<int>>& ans) {
    if (target == 0) {
        ans.push_back(cur);
        return;
    }
    for (int i = start; i < a.size(); i++) {
        if (a[i] > target) continue;
        cur.push_back(a[i]);
        dfs(i, target - a[i], a, cur, ans);
        cur.pop_back();
    }
}

vector<vector<int>> combinationSum(vector<int>& candidates, int target) {
    sort(candidates.begin(), candidates.end());
    vector<vector<int>> ans, cur;
    dfs(0, target, candidates, cur, ans);
    return ans;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 4 Generate Parentheses

Pattern: Backtracking

LeetCode: #22

■ Input	n pairs of parentheses
■ Output	all valid combinations
■ Constraints	$1 \leq n \leq 8$
■ Main Idea / Pattern	Add '(' while available. Add ')' only when it cannot make the prefix invalid.
■ Time Complexity	$O(4^n / \sqrt{n})$
■ Space Complexity	$O(n)$

■ Optimal C++ Code

```
void dfs(int open, int close, int n, string& cur, vector<string>& ans) {
    if (cur.size() == 2 * n) {
        ans.push_back(cur);
        return;
    }

    if (open < n) {
        cur += '(';
        dfs(open + 1, close, n, cur, ans);
        cur.pop_back();
    }

    if (close < open) {
        cur += ')';
        dfs(open, close + 1, n, cur, ans);
        cur.pop_back();
    }
}

vector<string> generateParenthesis(int n) {
    vector<string> ans;
    string cur;
    dfs(0, 0, n, cur, ans);
    return ans;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 5 N-Queens

Pattern: Backtracking

LeetCode: #51

■ Input	integer n
■ Output	all valid queen placements
■ Constraints	$1 \leq n \leq 9$
■ Main Idea / Pattern	Place one queen per row and reject columns and diagonals already occupied.
■■ Time Complexity	$O(n!)$
■ Space Complexity	$O(n)$

■ Optimal C++ Code

```
void dfs(int r, int n, vector<string>& board,
        vector<bool>& col, vector<bool>& d1, vector<bool>& d2,
        vector<vector<string>>& ans) {
    if (r == n) {
        ans.push_back(board);
        return;
    }
    for (int c = 0; c < n; c++) {
        if (col[c] || d1[r - c + n] || d2[r + c])
            continue;

        col[c] = d1[r - c + n] = d2[r + c] = true;
        board[r][c] = 'Q';

        dfs(r + 1, n, board, col, d1, d2, ans);

        board[r][c] = '.';
        col[c] = d1[r - c + n] = d2[r + c] = false;
    }
}

vector<vector<string>> solveNQueens(int n) {
    vector<vector<string>> ans;
    vector<string> board(n, string(n, '.'));
    vector<bool> col(n), d1(2 * n), d2(2 * n);
    dfs(0, n, board, col, d1, d2, ans);
    return ans;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 6 Word Search

Pattern: Backtracking on Grid

LeetCode: #79

■ Input	character grid and word
■ Output	whether word exists in the grid
■ Constraints	grid dimensions bounded
■ Main Idea / Pattern	Try each matching cell, mark it visited temporarily, explore four directions, then undo.
■ Time Complexity	$O(mn \cdot 4^L)$
■ Space Complexity	$O(L)$

■ Optimal C++ Code

```
bool dfs(int r, int c, int i, vector<vector<char>>& b, string& w) {
    if (i == w.size()) return true;
    if (r < 0 || r >= b.size() || c < 0 || c >= b[0].size() ||
        b[r][c] != w[i])
        return false;
    char save = b[r][c];
    b[r][c] = '#';
    bool ok = dfs(r + 1, c, i + 1, b, w) ||
              dfs(r - 1, c, i + 1, b, w) ||
              dfs(r, c + 1, i + 1, b, w) ||
              dfs(r, c - 1, i + 1, b, w);
    b[r][c] = save;
    return ok;
}

bool exist(vector<vector<char>>& board, string word) {
    for (int r = 0; r < board.size(); r++)
        for (int c = 0; c < board[0].size(); c++)
            if (dfs(r, c, 0, board, word))
                return true;
    return false;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 7 Subsets

Pattern: Backtracking

LeetCode: 78

Input	nums
Output	all subsets
Constraints	$0 \leq \text{nums.size()} \leq 10$
Main Idea / Pattern	At each index choose to include the value or skip it.
Time Complexity	$O(n * 2^n)$
Space Complexity	$O(n)$ recursion, excluding output

Optimal C++ Code

```
class Solution {
    vector<vector<int>> ans;
    void backtrack(vector<int>& nums, int i, vector<int>& cur) {
        if (i == nums.size()) {
            ans.push_back(cur);
            return;
        }
        backtrack(nums, i + 1, cur);
        cur.push_back(nums[i]);
        backtrack(nums, i + 1, cur);
        cur.pop_back();
    }
public:
    vector<vector<int>> subsets(vector<int>& nums) {
        vector<int> cur;
        backtrack(nums, 0, cur);
        return ans;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 8 Permutations

Pattern: Backtracking

LeetCode: 46

Input	nums
Output	all permutations
Constraints	1 <= nums.size() <= 6; values distinct
Main Idea / Pattern	Choose one unused value, recurse, then undo the choice.
Time Complexity	O(n * n!)
Space Complexity	O(n)

Optimal C++ Code

```
class Solution {
    vector<vector<int>>> ans;

    void backtrack(vector<int>& nums, vector<int>& cur, vector<bool>& used) {
        if (cur.size() == nums.size()) {
            ans.push_back(cur);
            return;
        }

        for (int i = 0; i < nums.size(); i++) {
            if (used[i]) continue;

            used[i] = true;
            cur.push_back(nums[i]);
            backtrack(nums, cur, used);

            cur.pop_back();
            used[i] = false;
        }
    }

public:
    vector<vector<int>>> permute(vector<int>& nums) {
        vector<int> cur;
        vector<bool> used(nums.size(), false);
        backtrack(nums, cur, used);
        return ans;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 9 Combination Sum

Pattern: Backtracking

LeetCode: 39

Input	candidates, target
Output	all combinations summing to target
Constraints	candidates.size() <= 30; target <= 40
Main Idea / Pattern	Choose a candidate, recurse without advancing to allow reuse, then undo.
Time Complexity	Exponential
Space Complexity	O(target) recursion, excluding output

Optimal C++ Code

```
class Solution {
    vector<vector<int>> ans;
    void backtrack(vector<int>& a, int start, int target, vector<int>& cur) {
        if (target == 0) {
            ans.push_back(cur);
            return;
        }
        for (int i = start; i < a.size(); i++) {
            if (a[i] > target) continue;
            cur.push_back(a[i]);
            backtrack(a, i, target - a[i], cur);
            cur.pop_back();
        }
    }
public:
    vector<vector<int>> combinationSum(vector<int>& candidates, int target) {
        vector<int> cur;
        backtrack(candidates, 0, target, cur);
        return ans;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 10 Letter Combinations of a Phone Number

Pattern: Choice Tree Backtracking

LeetCode: 17

Input	digits
Output	all possible letter combinations
Constraints	0 <= digits.size() <= 4
Main Idea / Pattern	Each digit creates a set of choices; recurse through one digit at a time and undo.
Time Complexity	$O(4^n * n)$
Space Complexity	$O(n)$

Optimal C++ Code

```
class Solution {
    vector<string> key = {
        "", "", "abc", "def", "ghi", "jkl",
        "mno", "pqrs", "tuv", "wxyz"
    };

    void backtrack(string& digits, int i, string& cur, vector<string>& ans) {
        if (i == digits.size()) {
            ans.push_back(cur);
            return;
        }

        for (char c : key[digits[i] - '0']) {
            cur.push_back(c);
            backtrack(digits, i + 1, cur, ans);
            cur.pop_back();
        }
    }

public:
    vector<string> letterCombinations(string digits) {
        if (digits.empty()) return {};

        vector<string> ans;
        string cur;
        backtrack(digits, 0, cur, ans);
        return ans;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.